

EuMINE DataBridge Hackathon 2026

Database Access Guide

Materials Project • JARVIS-DFT • AFLOW • NOMAD • OQMD

EuMINE Hackathon Organizing Committee

May 2026 — v1.0

Contents

1	Materials Project	2
1.1	Overview	2
1.2	Getting an API key	2
1.3	Installation	2
1.4	Basic query examples	3
1.5	Rate limits and quotas	3
2	JARVIS-DFT	4
2.1	Overview	4
2.2	Installation	4
2.3	Bulk download (recommended)	4
2.4	Available datasets	4
2.5	Accessing structures	4
3	AFLOW	5
3.1	Overview	5
3.2	REST API base URL	5
3.3	Query syntax	5
3.4	Key field names	6
3.5	CIF download	6
3.6	Rate limits	6
4	NOMAD	6
4.1	Overview	6
4.2	REST API base URL	6
4.3	Query for formation energy (Python)	6
4.4	Pagination	7
4.5	Key field names	7
4.6	Rate limits	8
5	OQMD	8
5.1	Overview	8
5.2	REST API base URL	8
5.3	Query examples	8
5.4	Key field names	9
5.5	Rate limits	9

6	Common Pitfalls and Best Practices	9
6.1	API rate limits	9
6.2	Pagination	9
6.3	Property name differences	9
6.4	Missing and invalid values	10
6.5	Functional and methodological biases	10
6.6	Formula and structure matching	10
6.7	Storing large datasets efficiently	10
	Summary: Quick Reference	11

This guide explains how to download and query each of the five materials databases supported in the EuMINE DataBridge Hackathon. You are required to use **at least two** of them in your submission. All databases are publicly accessible free of charge; some require a (free) account for API access.

Python environment required:

```
1 conda create -n eumine-hackathon python=3.11
2 conda activate eumine-hackathon
3 pip install mp-api pymatgen jarvis-tools requests pandas numpy
```

1 Materials Project

1.1 Overview

The Materials Project (<https://materialsproject.org>) provides DFT-computed properties (PBE functional) for ~150 000 inorganic materials, including formation energies, band gaps, elastic constants, and crystal structures.

1.2 Getting an API key

1. Go to <https://materialsproject.org> and click **Sign in**.
2. Create a free account (Google or email).
3. Navigate to **Dashboard** → **API key**.
4. Copy your key (a 32-character string).
5. Store it as an environment variable — *never* hard-code it in scripts:

```
1 export MP_API_KEY="your_key_here"
2 # or add to ~/.bashrc / ~/.zshrc
```

1.3 Installation

```
1 pip install mp-api pymatgen
```

1.4 Basic query examples

Query formation energy and band gap (no structures):

```

1 import os
2 from mp_api.client import MPRester
3
4 with MPRester(os.environ["MP_API_KEY"]) as mpr:
5     docs = mpr.materials.summary.search(
6         has_props=["thermo", "electronic_structure"],
7         fields=["material_id", "formula_pretty",
8               "formation_energy_per_atom", "band_gap",
9               "nsites", "nelements", "symmetry"],
10        chunk_size=1000,
11        num_chunks=None,
12    )
13    for doc in docs:
14        if doc.formation_energy_per_atom is None or doc.band_gap is None:
15            continue
16        print(doc.material_id, doc.formation_energy_per_atom, doc.
              band_gap)

```

Fetch a single structure by material ID:

```

1 from mp_api.client import MPRester
2 import os
3
4 with MPRester(os.environ["MP_API_KEY"]) as mpr:
5     structure = mpr.get_structure_by_material_id("mp-22862") # NaCl
6 print(structure) # pymatgen Structure object
7 structure.to(fmt="cif", filename="NaCl.cif")

```

Batch fetch structures for a list of IDs:

```

1 from mp_api.client import MPRester
2 import os
3
4 ids = ["mp-22862", "mp-13", "mp-2657"] # NaCl, Fe, TiO2
5
6 with MPRester(os.environ["MP_API_KEY"]) as mpr:
7     structures = mpr.get_structures(ids)
8
9 for mid, struct in zip(ids, structures):
10    struct.to(fmt="cif", filename=f"{mid}.cif")

```

1.5 Rate limits and quotas

- Free tier: **30 requests/minute**, no daily cap.
- Use `chunk_size=1000` and `num_chunks=None` to stream large queries without hitting per-request limits.
- Avoid fetching `structure` in bulk summary queries — it multiplies bandwidth by 100×. Fetch structures separately for only the materials you need.

2 JARVIS-DFT

2.1 Overview

JARVIS-DFT (<https://jarvis.nist.gov>) provides DFT-computed (OptB88vdW functional) properties for $\sim 75\,000$ materials with strong coverage of 2D and van-der-Waals materials. Data is distributed as bulk JSON downloads via Figshare — no API key required.

2.2 Installation

```
1 pip install jarvis-tools
```

2.3 Bulk download (recommended)

```
1 from jarvis.db.figshare import data as jdata
2
3 # Downloads ~400 MB once; cached in ~/.jarvis/ for all subsequent calls
4 dft_3d = jdata("dft_3d")
5 print(f"Total entries: {len(dft_3d)}") # ~75 000
6
7 for entry in dft_3d:
8     jid = entry.get("jid", "")
9     formula = entry.get("formula", "")
10    ef = entry.get("formation_energy_peratom")
11    bg = entry.get("optb88vdw_bandgap")
12    natoms = entry.get("natoms")
13    spg = entry.get("spg_number") or entry.get("spacegroup_number")
14    print(jid, formula, ef, bg)
```

2.4 Available datasets

Other datasets accessible via the same interface:

Dataset name	Description
dft_3d	3D bulk materials (main dataset)
dft_2d	2D monolayers
cfid_3d	Classical force-field inspired descriptors
jff	JARVIS force-field data

2.5 Accessing structures

JARVIS entries include atomic positions as a nested dictionary compatible with `jarvis.core.atoms.Atoms`:

```
1 from jarvis.core.atoms import Atoms
2 from jarvis.io.vasp.inputs import Poscar
3
4 entry = dft_3d[0]
5 atoms = Atoms.from_dict(entry["atoms"])
6 Poscar(atoms).write_file("structure.vasp") # write as POSCAR
```

Convert to pymatgen Structure:

```
1 from pymatgen.io.vasp import Poscar as pmgPoscar
2
3 struct = pmgPoscar.from_file("structure.vasp").structure
4 struct.to(fmt="cif", filename="structure.cif")
```

3 AFLOW

3.1 Overview

AFLOW (<https://aflow.org>) is one of the largest repositories of high-throughput DFT calculations, with ~3.5 million entries across multiple DFT flavors. Access is via a REST API — no account required.

3.2 REST API base URL

<http://aflow.org/API/aflowlib.json?>

3.3 Query syntax

Parameters are appended as **key=value** pairs separated by **&**.

Retrieve entries with both formation energy and band gap:

```
1 curl "http://aflow.org/API/aflowlib.json?catalog=icsd&paging=1&\
2 Egap(*),enthalpy_formation_atom(*),compound(*),auid(*)&format=json" \
3 | python -m json.tool | head -80
```

Python — paginated download:

```
1 import requests, json, time
2
3 BASE = "http://aflow.org/API/aflowlib.json"
4 FIELDS = "auid,compound,enthalpy_formation_atom,EGap,spacegroup_relax"
5 PAGE_SIZE = 200
6
7 results = []
8 page = 1
9 while True:
10     url = (f"{BASE}?catalog=icsd&paging={page}&"
11           f"{FIELDS.replace(',', '_')('*),'_+_'('*),'_'}&format=json")
12     r = requests.get(url, timeout=60)
13     if r.status_code != 200 or not r.text.strip():
14         break
15     data = r.json()
16     if not data:
17         break
18     results.extend(data)
19     page += 1
20     time.sleep(0.5)    # be polite --- 2 req/s is safe
21
22 print(f"Retrieved {len(results)} AFLOW entries")
```

3.4 Key field names

AFLOW field	Property	Unit
enthalpy_formation_atom	Formation energy per atom	eV/atom
Egap	Band gap (GGA)	eV
Egap_type	Gap type (direct/indirect)	—
compound	Chemical formula	—
spacegroup_relax	Space group number (relaxed)	—
natoms	Number of atoms in cell	—

3.5 CIF download

```

1 import requests
2
3 auid = "aflow:some_auid_string"
4 # Replace : with / in AUID to build the file path
5 path = auid.replace("aflow:", "").replace(":", "/")
6 url = f"http://aflow.org/{path}?files=CONTCAR.relax.vasp"
7 r = requests.get(url, timeout=30)
8 with open("structure.vasp", "wb") as f:
9     f.write(r.content)

```

3.6 Rate limits

AFLOW has no documented rate limit but imposes soft throttling. Keep requests below **2/second**; use `time.sleep(0.5)` between calls.

4 NOMAD

4.1 Overview

NOMAD (<https://nomad-lab.eu>) aggregates DFT and experimental data from many codes and groups worldwide, with rich metadata and a powerful REST API. No account required for read access; a (free) NOMAD account enables uploads.

4.2 REST API base URL

<https://nomad-lab.eu/prod/v1/api/v1/>

OpenAPI documentation: <https://nomad-lab.eu/prod/v1/api/v1/extensions/docs>

4.3 Query for formation energy (Python)

```

1 import requests, json
2
3 BASE = "https://nomad-lab.eu/prod/v1/api/v1"
4
5 query = {
6     "query": {
7         "and": [
8             {"results.material.structural_type": "bulk"},
9             {"results.properties.electronic.band_structure_electronic:
              exists": True},

```

```

10     ]
11 },
12 "required": {
13     "results": {
14         "material": {"chemical_formula_reduced": "*"},
15         "symmetry": {"space_group_number": "*"}},
16     "properties": {
17         "thermodynamic": {"formation_energy": "*"},
18         "electronic": {"band_gap": "*"},
19     }
20 }
21 },
22 "pagination": {"page_size": 100}
23 }
24
25 r = requests.post(f"{BASE}/entries/query", json=query, timeout=60)
26 data = r.json()
27
28 for entry in data.get("data", []):
29     mat = entry["results"]["material"]
30     prop = entry["results"]["properties"]
31     ef = prop.get("thermodynamic", {}).get("formation_energy")
32     bg = prop.get("electronic", {}).get("band_gap")
33     formula = mat.get("chemical_formula_reduced")
34     spg = mat.get("symmetry", {}).get("space_group_number")
35     print(formula, spg, ef, bg)

```

4.4 Pagination

NOMAD uses cursor-based pagination. To iterate all pages:

```

1 all_entries = []
2 query["pagination"] = {"page_size": 100}
3 next_url = f"{BASE}/entries/query"
4
5 while next_url:
6     r = requests.post(next_url, json=query, timeout=60)
7     data = r.json()
8     all_entries.extend(data.get("data", []))
9     next_page = data.get("pagination", {}).get("next_page_after_value")
10    if next_page:
11        query["pagination"]["page_after_value"] = next_page
12    else:
13        break

```

4.5 Key field names

NOMAD path	Property
results.properties.thermodynamic.formation_energy	Formation energy (eV/atom)
results.properties.electronic.band_gap	Band gap (eV)
results.material.chemical_formula_reduced	Reduced formula
results.material.symmetry.space_group_number	Space group

4.6 Rate limits

NOMAD enforces **10 req/s** for anonymous users; use `page_size=100` to minimise the number of requests.

5 OQMD

5.1 Overview

The Open Quantum Materials Database (<https://oqmd.org>) contains ~ 1 million DFT entries computed with VASP/PBE. Access is via a public REST API — no account required.

5.2 REST API base URL

<http://oqmd.org/oqmdapi/formationenergy?>

5.3 Query examples

Filter by element and request key properties:

```
1 curl "http://oqmd.org/oqmdapi/formationenergy?\n2 filter=element_set=(Ti,0)&\n3 fields=name,delta_e,band_gap,spacegroup,natoms&\n4 limit=50&offset=0&format=json"
```

Python — paginated download:

```
1 import requests, time\n2\n3 BASE = "http://oqmd.org/oqmdapi/formationenergy"\n4 PARAMS = {\n5     "fields": "name,delta_e,band_gap,spacegroup,natoms,unit_cell",\n6     "limit": 100,\n7     "offset": 0,\n8     "format": "json",\n9 }\n10\n11 results = []\n12 while True:\n13     r = requests.get(BASE, params=PARAMS, timeout=60)\n14     data = r.json()\n15     batch = data.get("data", [])\n16     if not batch:\n17         break\n18     results.extend(batch)\n19     PARAMS["offset"] += PARAMS["limit"]\n20     time.sleep(0.5)\n21     if PARAMS["offset"] > data.get("meta", {}).get("count", 0):\n22         break\n23\n24 print(f"Retrieved {len(results)} OQMD entries")
```

5.4 Key field names

OQMD field	Property	Unit
<code>name</code>	Chemical formula	—
<code>delta_e</code>	Formation enthalpy/atom	eV/atom
<code>band_gap</code>	Band gap	eV
<code>spacegroup</code>	Space group symbol	—
<code>natoms</code>	Atoms in cell	—

5.5 Rate limits

OQMD has no documented rate limit. Keep requests below **1/second** to avoid triggering the server's soft throttle.

6 Common Pitfalls and Best Practices

6.1 API rate limits

- **Materials Project:** 30 req/min — use the official client (`mp-api`) which handles chunking automatically.
- **JARVIS:** no API; single bulk download. Run it once, cache the result.
- **AFLOW:** no documented limit; stay at ≤ 2 req/s.
- **NOMAD:** 10 req/s; always use large `page_size`.
- **OQMD:** polite limit; 1 req/s recommended.

6.2 Pagination

All REST APIs return a limited number of results per call. Always implement pagination and check for empty responses. A common pattern:

```

1 offset = 0
2 while True:
3     batch = fetch_page(offset=offset, limit=PAGE_SIZE)
4     if not batch:
5         break
6     process(batch)
7     offset += PAGE_SIZE

```

6.3 Property name differences

The same quantity has different field names across databases. Use the following mapping when harmonising data:

Database	Formation energy field	Band gap field	Unit
Materials Project	<code>formation_energy_per_atom</code>	<code>band_gap</code>	eV, eV
JARVIS-DFT	<code>formation_energy_peratom</code>	<code>optb88vdw_bandgap</code>	eV, eV
AFLOW	<code>enthalpy_formation_atom</code>	<code>Egap</code>	eV, eV
NOMAD	<code>formation_energy</code>	<code>band_gap</code>	eV/atom, eV
OQMD	<code>delta_e</code>	<code>band_gap</code>	eV/atom, eV

Note: Always check units per database. NOMAD's `formation_energy` is sometimes in eV for the whole cell, not per atom — verify by dividing by `natoms` and cross-checking with a known compound.

6.4 Missing and invalid values

Databases use different sentinels for missing data:

```

1 MISSING = {None, "na", "Na", "NA", "", "None", "N/A", float("nan")}
2
3 def is_valid(value):
4     if value in MISSING:
5         return False
6     try:
7         return not (float(value) != float(value))    # NaN check
8     except (TypeError, ValueError):
9         return False

```

6.5 Functional and methodological biases

The same compound can yield very different values in different databases because the DFT functional, pseudopotentials, and convergence settings differ:

Database	Primary functional
Materials Project	PBE (GGA)
JARVIS-DFT	OptB88vdW (vdW-corrected GGA)
AFLOW	PBE (GGA), LDA for some
NOMAD	Mixed (source-dependent)
OQMD	PBE (GGA)

Band gaps are **systematically underestimated** by standard GGA. JARVIS reports slightly higher band gaps than MP for the same compound due to the OptB88vdW correction, especially for weakly bonded and layered systems. Document which sources you used and how you resolved discrepancies in your *Data Integration Report*.

6.6 Formula and structure matching

When linking entries across databases:

- Normalize all formulas to **reduced composition** (e.g., $\text{Na}_2\text{Cl}_2 \rightarrow \text{NaCl}$). Use `pymatgen.core.Composition`.
- Match on **reduced formula + space group number** as a primary key.
- Apply an **nsites tolerance of ± 2** to reconcile slight supercell differences.
- Deduplicate after merging (keep one MP entry and one JARVIS entry per bridge material to avoid data leakage).

6.7 Storing large datasets efficiently

- Use **JSONL** (one JSON object per line) for streaming writes — safe against RAM overflow and easy to append to.
- Avoid loading all 150,000 MP entries into a Python list at once.
- For the hackathon, the provided Bridge Dataset ($\sim 1,000$ entries) is small enough to hold in memory as a Pandas DataFrame.

Summary: Quick Reference

Database	Size	API type	Auth	Install
Materials Project	~150k	Python client	API key (free)	<code>pip install mp-api</code>
JARVIS-DFT	~75k	Bulk download	None	<code>pip install jarvis-tools</code>
AFLOW	~3.5M	REST/JSON	None	<code>pip install requests</code>
NOMAD	Diverse	REST/JSON	None (read)	<code>pip install requests</code>
OQMD	~1M	REST/JSON	None	<code>pip install requests</code>

EuMINE DataBridge Hackathon 2026

hackathon@eumine.eu

Questions about data access? Post in #data-sources on Slack or open a GitHub Discussion at https://github.com/EuMINE-COST/eumine_hackaton_2026/discussions.